

Project: Designing a Telemetry and UX Database for
Chocolate-Doom Research
Part A: E-R, Data Dictionary

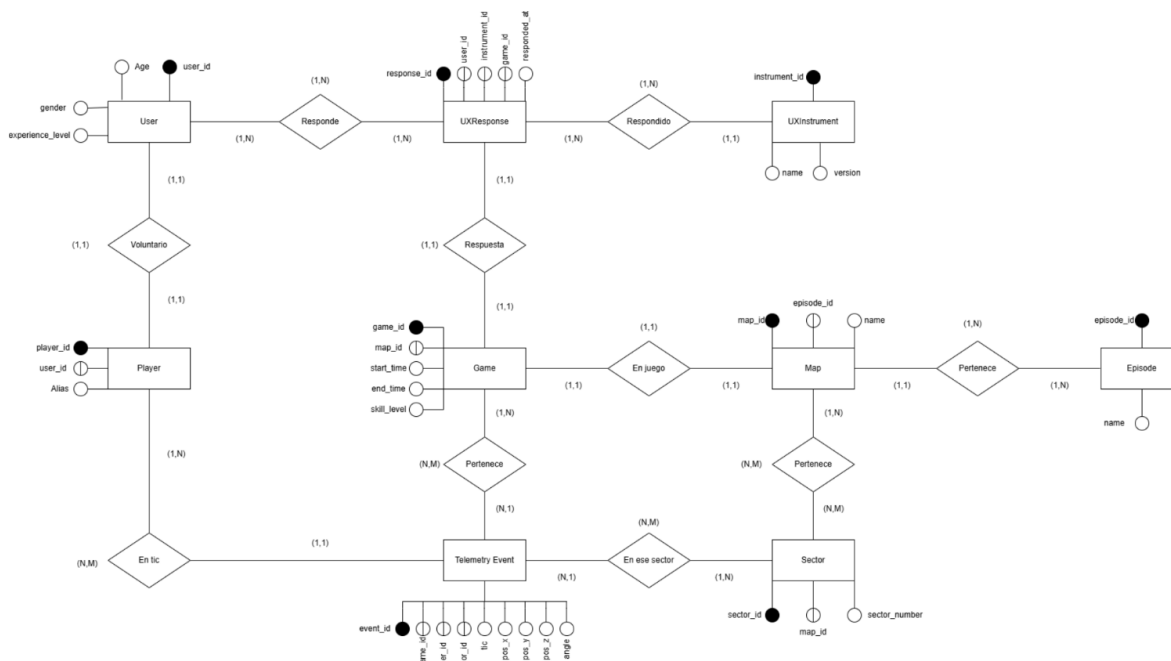
DBS – Semester Project
May 2026

Introducción

Este documento presenta el diseño de la base de datos para el análisis de telemetría y comportamiento en *Chocolate-Doom*. El objetivo es estructurar los datos capturados por *tic* y vincularlos con el perfil demográfico de los jugadores y sus resultados en las encuestas de experiencia de usuario BANGS.

A través del Diagrama Entidad-Relación, se define la jerarquía entre sesiones, mapas y registros granulares de telemetría, mientras que el Diccionario de Datos especifica las restricciones técnicas necesarias para garantizar la calidad de la información y la ética en el manejo de datos de los voluntarios.

E-R Diagrama



Diccionario

User

Tupla	Tipo	Clave	Condiciones	Descripción
user_id	Integer	PK	No Null	Identidad en el juego
age	Integer		age >= 0	Nivel de experiencia en videojuegos

gender	Varchar(20)			Género declarado
experience_level	Varchar(20)			Nivel de experiencia en videojuegos

Tupla	Tipo	Clave	Descripción
player_id	Integer	PK	Identidad en el juego
user_id	Integer	FK a user	Víncula al voluntario
alias	Varchar(50)		Nombre en el juego

Player

Episode

Tupla	Tipo	Clave	Condiciones	Descripción
episode_id	Integer	PK	No Null	Número de episodio
name	Varchar(50)			Nombre del episodio

Map

Tupla	Tipo	Clave	Condiciones	Descripción
map_id	Integer	PK	No Null	ID único del mapa
episode_id	Integer	FK a Episode	No Null	Episodio al que pertenece
name	Varchar(50)			Nombre del mapa

Sector

Tupla	Tipo	Clave	Condiciones	Descripción
sector_id	Integer	PK	No Null	ID del sector
map_id	Integer	FK a Map	No Null	Mapa al que pertenece
sector_number	Integer			Número de sector en el mapa

Game

Tupla	Tipo	Clave	Condiciones	Descripción
-------	------	-------	-------------	-------------

game_id	Integer	PK	No Null	Sesión de juego única
map_id	Integer	FK a Map	No Null	Mapa jugado
start_time	TIME		No Null	Inicio de la sesión
end_time	TIME		end_time > start_time	Fin de la sesión
skill_level	Integer			Dificultad seleccionada

Telemetry Event

Tupla	Tipo	Clave	Condiciones	Descripción
event_id	Integer	PK	No Null	ID del evento de telemetría
game_id	Integer	FK a game	No Null	Sesión a la que pertenece
player_id	Integer	FK a player	No Null	Jugador en ese tic
tic	INTEGER	No Null, CHECK > 0		Número de tic del motor
pos_x	NUMERIC(10,2)	NOT NULL		Posición X del jugador
pos_y	NUMERIC(10,2)	NOT NULL		Posición Y del jugador
pos_z	NUMERIC(10,2)	NOT NULL		Posición Z del jugador
angle	NUMERIC(6,2)	NOT NULL		Ángulo de visión (grados)

UXInstrument

Tupla	Tipo	Clave	Condiciones	Descripción
instrument_id	Integer	PK	No Null	ID del instrumento (ej: BANGS)
name	Varchar(20)			
version	Varchar(10)			

UXResponse

Tupla	Tipo	Clave	Condiciones	Descripción
response_id	Integer	PK	No Null	Respuesta de un usuario al instrumento
user_id	Integer	FK a user	No Null	Usuario que respondió
instrument_id	Integer	FK a instrument	No Null	Instrumento respondido
game_id	Integer	FK a game		Sesión después de la cual respondió
responded_at	TIME			Momento de respuesta

UXResponseItem

Tupla	Tipo	Clave	Condiciones	Descripción
response_id	Integer	FK a UXResponse	No Null	Respuesta de un usuario al instrumento
item_id	Integer	FK a UXItem	No Null	Pregunta que se responde
score	Integer	CHECK (1-5)	No Null	Instrumento respondido

UXItem

Tupla	Tipo	Clave	Condiciones	Descripción
item_id	Integer	PK	No Null	Pregunta
instrument_id	Integer	FK a UXInstrument	No Null	Pregunta que se responde
item_number	Integer			
question_text	Varchar(200)			

6. ETL Description

6.1 Pipeline

El pipeline de ingreso sigue tres etapas:

1. Carga raw: Los archivos TSV generados por el motor se cargan a una tabla staging_telemetry donde todos los campos son TEXT.
2. Validacion: Se identifican y registran en error_log_telemetry las filas con valores nulos, no numericos o fuera de rango.
3. Transformacion: Las filas validas se castean a sus tipos correctos, se duplican por (game_id, tic, player_id) y se insertan en TelemetryEvent.

6.2 Muestra de Telemetria Real

A continuacion se muestra una muestra de los datos capturados jugando Chocolate-Doom:

timestamp	tic	x	y	z	angle	momx	momy
2026-05-20 15:11:51	188	0	- 160	0	90.00	-1	1
2026-05-20 15:11:52	223	- 126	-31	0	80.16	-7	-3
2026-05-20 15:11:53	258	-77	-46	0	97.03	9	-1
2026-05-20 15:11:54	293	47	- 175	0	101.25	-1	2
2026-05-20 15:11:55	328	3	106	0	104.06	1	-7

Las sesiones reales capturadas se encuentran en la carpeta data/ del repositorio (sesion1.tsv, sesion2.tsv, sesion3.tsv, sesion4.tsv).

```
- Doom Telemetry\q
      SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public';
  table_name
-----
User
player
episode
map
uxinstrument
uxitem
sector
game
telemetryevent
uxresponse
uxresponseitem
11 rows)

Doom_telemetry=# SELECT * FROM player;
 player_id | user_id |  alias
-----+-----+-----
          1 |         1 | DoomGuy01
          2 |         2 | Slayer02
          3 |         3 | Rip03
          4 |         4 | Tear04
          5 |         5 | Frag05
          6 |         6 | Raze06
6 rows)
```

```
doom_telemetry=# SELECT 'Episode'          AS tabla, COUNT(*) AS filas FROM Episode
UNION ALL
SELECT 'Map',          COUNT(*) FROM Map
UNION ALL
SELECT 'Sector',       COUNT(*) FROM Sector
UNION ALL
SELECT '"User"',       COUNT(*) FROM "User"
UNION ALL
SELECT 'Player',       COUNT(*) FROM Player
UNION ALL
SELECT 'Game',         COUNT(*) FROM Game
UNION ALL
SELECT 'TelemetryEvent',COUNT(*) FROM TelemetryEvent
UNION ALL
SELECT 'UXInstrument',  COUNT(*) FROM UXInstrument
UNION ALL
SELECT 'UXItem',       COUNT(*) FROM UXItem
UNION ALL
SELECT 'UXResponse',   COUNT(*) FROM UXResponse
UNION ALL
SELECT 'UXResponseItem',COUNT(*) FROM UXResponseItem;
      tabla      | filas
-----+-----
Episode          |      3
Map               |      3
Sector           |      9
"User"           |      6
Player           |      6
Game             |      6
TelemetryEvent   |  20034
UXInstrument      |       1
UXItem           |       0
UXResponse       |       0
UXResponseItem   |       0
(11 rows)

doom_telemetry=#
```

Indexes:

A continuación el código usado para crear los 3 indexes basados en el enunciado del proyecto:

```
doom_telemetry=# CREATE INDEX IF NOT EXISTS idx_telemetry_game_player_tic
ON TelemetryEvent (game_id, player_id, tic)
INCLUDE (pos_x, pos_y, pos_z);
```

Teniendo en cuenta la diferencia de localización de atributos en la organización de nuestro schema, 3 indexes intentan reflejar el 2do index sugerido:

```
doom_telemetry=# CREATE INDEX IF NOT EXISTS idx_map_episode_id ON Map (episode_id);
doom_telemetry=# CREATE INDEX IF NOT EXISTS idx_sector_map_id ON Sector (map_id);
doom_telemetry=# CREATE INDEX IF NOT EXISTS idx_game_map_id ON Game (map_id);
```

El primer y tercer index utilizan include, esto debido a que los indexes iniciales resultaban en el mismo o inclusive un mayor tiempo de ejecución comparado a si el

schema no tuviera indexes; por lo cual al incluir estos atributos a ignorar como carga extra durante la búsqueda del índice deseado en los nodulos del Btree por defecto, habilitamos un “escaneo” de únicamente índices que en retorno nos da un tiempo menor en ms.

```
doom_telemetry=# CREATE INDEX IF NOT EXISTS idx_telemetry_pos_x_y
ON TelemetryEvent (pos_x, pos_y)
INCLUDE (game_id, player_id, tic);
```

Query utilizada con EXPLAIN/ANALYZE:

```
doom_telemetry=# EXPLAIN ANALYZE
SELECT
e1.game_id,
e1.player_id AS jugador_a,
e2.player_id AS jugador_b,
COUNT(DISTINCT e1.tic) AS tics_juntos
FROM TelemetryEvent e1
JOIN TelemetryEvent e2
ON e1.game_id = e2.game_id
AND e1.tic = e2.tic
AND e1.player_id < e2.player_id
AND FLOOR(e1.pos_x / 250) = FLOOR(e2.pos_x / 250)
AND FLOOR(e1.pos_y / 250) = FLOOR(e2.pos_y / 250)
GROUP BY e1.game_id, e1.player_id, e2.player_id
ORDER BY tics_juntos DESC;
```

Resultado previo a la creación de índices:

```
QUERY PLAN
-----
Sort (cost=1664.27..1664.27 rows=1 width=20) (actual time=68.386..68.391 rows=0 loops=1)
  Sort Key: (count(DISTINCT e1.tic)) DESC
  Sort Method: quicksort  Memory: 25kB
  -> GroupAggregate (cost=1664.23..1664.26 rows=1 width=20) (actual time=68.347..68.351 rows=0 loops=1)
    Group Key: e1.game_id, e1.player_id, e2.player_id
    -> Sort (cost=1664.23..1664.24 rows=1 width=16) (actual time=68.343..68.347 rows=0 loops=1)
      Sort Key: e1.game_id, e1.player_id, e2.player_id
      Sort Method: quicksort  Memory: 25kB
      -> Hash Join (cost=782.02..1664.22 rows=1 width=16) (actual time=68.321..68.325 rows=0 loops=1)
        Hash Cond: ((e1.game_id = e2.game_id) AND (e1.tic = e2.tic) AND (floor((e1.pos_x / '250'::numeric)) = floor((e2.pos_x / '250'::numeric)) = floor((e2.pos_y / '250'::numeric)))
        Join Filter: (e1.player_id < e2.player_id)
        Rows Removed by Join Filter: 20034
        -> Seq Scan on telemetryevent e1 (cost=0.00..381.34 rows=20034 width=26) (actual time=0.047..1.554 rows=20034 loops=1)
        -> Hash (cost=381.34..381.34 rows=20034 width=26) (actual time=24.859..24.861 rows=20034 loops=1)
          Buckets: 32768  Batches: 1  Memory Usage: 1413kB
          -> Seq Scan on telemetryevent e2 (cost=0.00..381.34 rows=20034 width=26) (actual time=0.056..4.222 rows=20034 loops=1)
Planning Time: 0.642 ms
Execution Time: 68.554 ms
(18 rows)

(END)
```

Resultado después de la creación de los índices:

Diferencia minima en el tiempo de ejecución.

```

QUERY PLAN
-----
Sort (cost=2487.18..2487.19 rows=1 width=20) (actual time=62.994..63.000 rows=0 loops=1)
  Sort Key: (count(DISTINCT e1.tic)) DESC
  Sort Method: quicksort  Memory: 25kB
-> GroupAggregate (cost=2487.15..2487.17 rows=1 width=20) (actual time=62.989..62.994 rows=0 loops=1)
   Group Key: e1.game_id, e1.player_id, e2.player_id
   -> Sort (cost=2487.15..2487.15 rows=1 width=16) (actual time=62.986..62.991 rows=0 loops=1)
        Sort Key: e1.game_id, e1.player_id, e2.player_id
        Sort Method: quicksort  Memory: 25kB
        -> Hash Join (cost=1193.77..2487.14 rows=1 width=16) (actual time=62.979..62.984 rows=0 loops=1)
             Hash Cond: ((e1.game_id = e2.game_id) AND (e1.tic = e2.tic) AND (floor((e1.pos_x / '250'::numeric)) = floor((e2.pos_x / '250'::numeric)))
             Join Filter: (e1.player_id < e2.player_id)
             Rows Removed by Join Filter: 20034
             -> Index Only Scan using idx_telemetry_pos_x_y on telemetryevent e1 (cost=0.29..792.80 rows=20034 width=16) (actual time=0.000..0.000 rows=0 loops=1)
                  Heap Fetches: 0
             -> Hash (cost=792.80..792.80 rows=20034 width=26) (actual time=20.095..20.098 rows=20034 loops=1)
                  Buckets: 32768  Batches: 1  Memory Usage: 1413kB
                  -> Index Only Scan using idx_telemetry_pos_x_y on telemetryevent e2 (cost=0.29..792.80 rows=20034 width=16) (actual time=0.000..0.000 rows=0 loops=1)
                       Heap Fetches: 0
Planning Time: 0.393 ms
Execution Time: 63.069 ms
(20 rows)
(END)

```

Views:

VIEW 1 — v_avg_duration_per_map

```

SELECT map_id, AVG(end_time - start_time) AS avg_duration
FROM Game
GROUP BY map_id;
CREATE VIEW
doom_telemetry=# SELECT * FROM v_avg_duration_per_map;
 map_id | avg_duration
-----+-----
      3 | 00:05:29
      2 | 00:02:38
      1 | 00:09:25
(3 rows)

```

Esta vista calcula la duración promedio de las sesiones de juego agrupadas por mapa. Permite consultar rápidamente qué mapas retienen más tiempo a los jugadores sin necesidad de reescribir el AVG y el JOIN cada vez

VIEW 2 — v_ux_above_avg_players

```
doom_telemetry=# CREATE VIEW v_ux_above_avg_players AS
SELECT DISTINCT p.player_id, p.alias, ur.response_id, ur.responded_at
FROM Game g
JOIN TelemetryEvent te ON g.game_id = te.game_id
JOIN Player p ON te.player_id = p.player_id
JOIN UXResponse ur ON ur.user_id = p.user_id
WHERE (g.end_time - g.start_time) > (
    SELECT AVG(end_time - start_time) FROM Game
);
CREATE VIEW
```

```
doom_telemetry=# SELECT * FROM v_ux_above_avg_players;
 player_id |  alias  | response_id | responded_at
-----+-----+-----+-----
          1 | DoomGuy01 |          1 | 
          2 | Slayer02  |          2 | 
          3 | Rip03     |          3 | 
          4 | Tear04    |          4 | 
          5 | Frag05    |          5 | 
          6 | Raze06    |          6 | 
(6 rows)
```

Esta vista cruza las sesiones de juego con las respuestas UX, filtrando solo los jugadores cuya sesión duró más que el promedio global. Es útil para analizar si los jugadores más comprometidos tienen mejor experiencia de usuario.

MATERIALIZED VIEW — mv_sector_hotspot

```
doom_telemetry=# CREATE MATERIALIZED VIEW mv_sector_hotspot AS
SELECT
    m.episode_id,
    te.game_id,
    COUNT(*) AS visit_count
FROM TelemetryEvent te
JOIN Game g ON te.game_id = g.game_id
JOIN Map m ON g.map_id = m.map_id
GROUP BY m.episode_id, te.game_id
ORDER BY visit_count DESC;
ERROR:  relation "mv_sector_hotspot" already exists
```

```
doom_telemetry=# SELECT * FROM mv_sector_hotspot
doom_telemetry=# ;
 episode_id | game_id | visit_count
-----+-----+-----
          1 |        1 |          3339
          1 |        2 |          3339
          3 |        6 |          3339
          2 |        3 |          3339
          2 |        4 |          3339
          3 |        5 |          3339
(6 rows)
```

Esta vista materializada agrupa los eventos de telemetría por episodio y sesión para identificar las zonas más visitadas. Al ser materializada, el resultado se guarda físicamente en disco, lo que acelera las consultas sobre 20,000 filas. Se actualiza con `REFRESH MATERIALIZED VIEW mv_sector_hotspot` cuando llegan nuevos datos.

Queries:

```
SELECT
  m.map_id,
  m.name
  COUNT(g.game_id) AS num_sesiones,
  AVG(g.end_time - g.start_time) AS duracion_promedio,
  ROUND(AVG(EXTRACT(EPOCH FROM (g.end_time - g.start_time)))::numeric, 1)
  | | | | | | | | | | | | | | | | AS duracion_promedio_seg
FROM Game g
JOIN Map m ON m.map_id = g.map_id
GROUP BY m.map_id, m.name
ORDER BY duracion_promedio DESC;
```

Esta consulta analiza el rendimiento y la participación en los distintos mapas del juego. Su objetivo es calcular el número total de sesiones de juego (`num_sesiones`) que se han llevado a cabo en cada uno, así como determinar la duración promedio de estas partidas. El tiempo de juego se calcula mediante la diferencia entre el inicio y el fin de cada sesión, presentándose en su formato de intervalo original y también convertido a segundos exactos (`duracion_promedio_seg`). Finalmente, los resultados se ordenan de forma descendente para mostrar primero aquellos mapas en los que las partidas tienden a ser más largas.

```

SELECT
    e1.player_id AS jugador_a,
    e2.player_id AS jugador_b,
    COUNT(*)     AS tics_compartidos,
    ROUND(AVG(
        SQRT( POWER(e1.pos_x - e2.pos_x, 2)
              + POWER(e1.pos_y - e2.pos_y, 2)
              + POWER(e1.pos_z - e2.pos_z, 2) )
    )::numeric, 2) AS distancia_promedio
FROM TelemetryEvent e1
JOIN TelemetryEvent e2
    ON e1.game_id = e2.game_id
   AND e1.tic     = e2.tic
   AND e1.player_id < e2.player_id
GROUP BY e1.player_id, e2.player_id
ORDER BY distancia_promedio ASC
LIMIT 10;

```

Esta consulta evalúa la proximidad física entre los participantes durante el juego. Su objetivo es identificar a las parejas de jugadores (jugador_a y jugador_b) que se mantienen más cerca a lo largo de una misma partida. Para lograrlo, empareja los eventos de telemetría que coinciden en el mismo instante de tiempo y aplica la fórmula de distancia euclidiana en tres dimensiones sobre las coordenadas X, Y y Z para obtener una distancia promedio (distancia_promedio). Al final, los datos se ordenan de menor a mayor para destacar exclusivamente a los 10 pares de jugadores con la menor separación física.

```

4
5 WITH pasos AS (
6     SELECT
7         player_id,
8         game_id,
9         SQRT( POWER(pos_x - LAG(pos_x) OVER w, 2)
10             + POWER(pos_y - LAG(pos_y) OVER w, 2)
11             + POWER(pos_z - LAG(pos_z) OVER w, 2) ) AS dist_paso
12     FROM TelemetryEvent
13     WINDOW w AS (PARTITION BY player_id, game_id ORDER BY tic)
14 ),
15 trayectoria_por_juego AS (
16     SELECT player_id, game_id, COALESCE(SUM(dist_paso), 0) AS dist_total
17     FROM pasos
18     GROUP BY player_id, game_id
19 )
20 SELECT
21     player_id,
22     ROUND(MIN(dist_total)::numeric, 2) AS trayectoria_mas_corta,
23     ROUND(MAX(dist_total)::numeric, 2) AS trayectoria_mas_larga
24 FROM trayectoria_por_juego
25 GROUP BY player_id
26 ORDER BY player_id;
27

```

Esta consulta examina el desplazamiento individual para identificar los extremos de movilidad de cada usuario. Su objetivo es determinar la distancia total recorrida más corta (`trayectoria_mas_corta`) y la más larga (`trayectoria_mas_larga`) registrada por cada jugador en sus distintas partidas. Esto se consigue calculando la distancia entre puntos de telemetría consecutivos usando funciones analíticas para medir el avance paso a paso. Posteriormente, se suman estas pequeñas distancias para obtener el recorrido total por juego, agrupando finalmente la información para extraer y mostrar los valores mínimos y máximos por jugador.

```

WITH conteo_celdas AS (
    SELECT
        ep.episode_id,
        ep.name AS episode_name,
        m.map_id,
        m.name AS map_name,
        FLOOR(te.pos_x / 250)::int AS grid_x,
        FLOOR(te.pos_y / 250)::int AS grid_y,
        COUNT(*) AS visitas
    FROM TelemetryEvent te
    JOIN Game g ON g.game_id = te.game_id
    JOIN Map m ON m.map_id = g.map_id
    JOIN Episode ep ON ep.episode_id = m.episode_id
    GROUP BY ep.episode_id, ep.name, m.map_id, m.name, grid_x, grid_y
),
ranking AS (
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY episode_id, map_id
                            ORDER BY visitas DESC) AS rn
    FROM conteo_celdas
)
SELECT episode_id, episode_name, map_id, map_name,
       grid_x, grid_y, visitas
FROM ranking
WHERE rn = 1
ORDER BY episode_id, map_id;

```

Esta consulta mapea el tráfico dentro de los escenarios para detectar las zonas de mayor actividad o "puntos calientes". Su objetivo es encontrar el cuadrante exacto que recibe más visitas para cada combinación de episodio y mapa. Para ello, divide el espacio virtual en una cuadrícula de sectores de 250 por 250 unidades (`grid_x`, `grid_y`) y cuenta la cantidad de registros de telemetría que caen dentro de cada una (`visitas`). Mediante una función de clasificación, se le asigna un ranking a cada sector según su volumen de tráfico, filtrando los resultados para mostrar únicamente la cuadrícula número uno, es decir, la más transitada.

```
SELECT
    e1.game_id,
    e1.player_id AS jugador_a,
    e2.player_id AS jugador_b,
    COUNT(DISTINCT e1.tic) AS tics_juntos
FROM TelemetryEvent e1
JOIN TelemetryEvent e2
    ON e1.game_id = e2.game_id
    AND e1.tic = e2.tic
    AND e1.player_id < e2.player_id
    AND FLOOR(e1.pos_x / 250) = FLOOR(e2.pos_x / 250)
    AND FLOOR(e1.pos_y / 250) = FLOOR(e2.pos_y / 250)
GROUP BY e1.game_id, e1.player_id, e2.player_id
ORDER BY tics_juntos DESC;
```

Esta consulta busca cuantificar el tiempo real de coexistencia espacial entre pares de jugadores. Su objetivo es medir cuántos momentos exactos (`tics_juntos`) comparten dos usuarios dentro de la misma zona física del mapa durante una partida. Esto se evalúa cruzando los datos de telemetría para encontrar coincidencias simultáneas en las que ambos jugadores se ubican dentro del mismo sector exacto de la cuadrícula de 250x250 unidades. Los resultados se agrupan por partida y pareja de jugadores, ordenándose de mayor a menor para evidenciar a los dúos que más tiempo interactúan en el mismo cuadrante.

```

WITH pasos AS (
  SELECT
    player_id,
    SQRT( POWER(pos_x - LAG(pos_x) OVER w, 2)
      + POWER(pos_y - LAG(pos_y) OVER w, 2)
      + POWER(pos_z - LAG(pos_z) OVER w, 2) ) AS dist_paso
  FROM TelemetryEvent
  WINDOW w AS (PARTITION BY player_id, game_id ORDER BY tic)
)
SELECT
  player_id,
  COUNT(dist_paso) AS num_pasos,
  ROUND(SUM(dist_paso)::numeric, 2) AS distancia_total,
  ROUND((SUM(dist_paso) / NULLIF(COUNT(dist_paso), 0))::numeric, 4)
  AS velocidad_promedio_por_tic
FROM pasos
GROUP BY player_id
ORDER BY distancia_total DESC;

```

Esta consulta genera un balance del desempeño físico y la movilidad general de cada jugador. Su objetivo es extraer métricas globales a partir de todo su historial, calculando la cantidad de movimientos o pasos realizados (`num_pasos`), la distancia acumulada en su totalidad (`distancia_total`) y el ritmo de movimiento (`velocidad_promedio_por_tic`). Este perfil se construye midiendo la separación entre posiciones consecutivas y sumando esos tramos, para luego dividir el recorrido total entre el número de pasos. El reporte final se ordena de forma descendente para resaltar a los jugadores que han cubierto la mayor distancia global.

Conclusión

El modelo diseñado permite integrar con éxito el flujo masivo de datos técnicos de juego con las métricas psicométricas de los voluntarios. La estructura garantiza que cada movimiento en los ejes esté correctamente asociado a un contexto de mapa y sesión específico, facilitando la detección de patrones de cooperación o sabotaje. Con este esquema, la base de datos está lista para soportar la fase de analítica y consultas de tendencias de movimiento sin comprometer la integridad ni la privacidad de los participantes.